

Math Foundations C: Optimization Theory and Matrix Calculus for Neural Networks

A single merged treatment of Boyd–Vandenberghe Ch. 1, §3.1–3.2, §9.1–9.3,
Goodfellow Ch. 4 and §8.1–8.3, and the Matrix Cookbook §2

Prepared for Tom Koreslesjak · companion to `NEC_thesis_syllabus.md`

June 2026

Abstract

Everything trained in the rest of this syllabus — expert networks, sequence encoders, the gate — is trained by a stochastic first-order method on a non-convex objective. This chapter builds the theory that makes such training legible: gradient descent analyzed exactly on the quadratic model problem (where every phenomenon — step-size limits, conditioning, momentum, preconditioning — is visible in closed form), the stochastic correction for minibatch noise, momentum as a damped oscillator, Adam as bias-corrected per-coordinate preconditioning, the matrix calculus that powers backpropagation (one differential trick replaces a zoo of memorized identities), and the variance-propagation logic of initialization. The destination: by the end you can read a training configuration — optimizer, learning rate, schedule, batch size, clipping, weight decay — and explain what each knob does in the language of eigenvalues and variances, which is the level Module 6 assumes when backpropagation is derived in full.

Contents

C.1The landscape: convex, non-convex, and what “solved” means	3
C.2Gradient descent, analyzed exactly	3
C.2.1The model problem and the eigenmode decomposition	3
C.2.2Conditioning: the number that runs the show	4
C.3Stochastic gradients: optimization under measurement noise	5
C.4Momentum: optimization as a damped oscillator	6
C.5Adaptive methods: AdaGrad, RMSProp, Adam	7

C.6Matrix calculus: the differential method	8
C.7Initialization: variance propagation through depth	9
C.8Capstone: reading a training configuration like a physicist	10
C.9Exercises	11
Source map and what comes next	12

How this chapter was assembled (and what was cut)

Boyd–Vandenberghe supplies the convex-analysis spine, Goodfellow the neural-network corrections to it, and the Matrix Cookbook the calculus — though the chapter teaches the *differential method* that generates the Cookbook’s tables rather than the tables themselves. Cuts, with reasons:

- *Boyd’s duality, interior-point, and constrained-optimization apparatus.* The KKT slice this syllabus needs was already built in Module 4 §4.3 and is cross-referenced, not repeated. Likewise Goodfellow §4.4’s constrained-optimization detour.
- *Second-order methods at depth (Boyd §9.5 onward).* Newton’s method appeared, working, as IRLS in Module 4 §4.8; at neural-network scale exact Newton is off the table ($p \times p$ Hessians with p in the millions), so it survives here only as the ideal that preconditioning and Adam imperfectly imitate. L-BFGS gets one sentence.
- *Goodfellow §8.2’s cliffs and long-term dependencies.* Deferred to Module 7, where exploding gradients through recurrence are load-bearing rather than anecdotal.
- *Matrix Cookbook §2.1 (determinant derivatives).* One identity is stated for the road ($\partial \log |A| = A^{-\top}$); its workout belongs to Module 8, where $\log |\Sigma|$ terms appear in Gaussian mixture likelihoods. The Cookbook’s §8 (Gaussian moments) is, per the syllabus, a skim-now-return-later reference for Foundations D.
- *Nesterov acceleration theory.* The method is described and its rate stated; the proof machinery (estimate sequences) is omitted — the gain over heavy-ball momentum is modest in the stochastic regime where this pipeline lives.

One deliberate de-duplication against the syllabus itself: the syllabus lists the momentum-ODE and SGD-on-quadratic derivations under *both* Foundations C and Module 6. They are done *here*, once; the Module 6 chapter will cross-reference. The same policy holds for Adam’s bias correction (checkpoint [here](#), exercise [there](#)): derived [here](#), Exercise [C.3](#).

Notation

As before; new conventions for this chapter. The objective is $f : \mathbb{R}^p \rightarrow \mathbb{R}$, typically an average loss $f(\beta) = \frac{1}{N} \sum_i \mathcal{L}_i(\beta)$; iterates are β_t , step size (learning rate) η , minibatch gradient $\tilde{g}_t$. On the quadratic model problem, $A \succ 0$ is symmetric with eigenvalues $\lambda_{\min} = \lambda_1 \leq \dots \leq \lambda_p = \lambda_{\max}$

and condition number $\kappa = \lambda_{\max}/\lambda_{\min}$. Gradients are column vectors shaped like their variable ($\nabla_{\mathbf{W}}\mathcal{L}$ has the shape of $\mathbf{W}$) — the convention every deep-learning framework uses.

C.1 The landscape: convex, non-convex, and what “solved” means

Recall from Module 4 what convexity buys. A function f is convex when $f(\theta\mathbf{u} + (1 - \theta)\mathbf{v}) \leq \theta f(\mathbf{u}) + (1 - \theta)f(\mathbf{v})$; for twice-differentiable f , equivalently $\nabla^2 f \succeq 0$ everywhere. The two consequences that made Modules 4–5 comfortable: every local minimum is global, and first-order conditions ($\nabla f = 0$, or $0 \in \partial f$) are sufficient certificates. Every training objective so far — least squares, ridge, lasso, logistic and softmax regression with or without L_2 penalty — was convex in its parameters, which is why those chapters could speak of *the* solution.

Neural networks end that comfort, for reasons worth knowing precisely rather than vaguely. Compose one hidden layer, $f(\mathbf{W}_1, \mathbf{W}_2) = \mathcal{L}(\mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x}))$: even with convex $\mathcal{L}$ and the mildest nonlinearity σ , the composition is non-convex in $(\mathbf{W}_1, \mathbf{W}_2)$ — convexity survives affine pre-composition (Module 4, Exercise 4.6(d)) but not nonlinear pre-composition. Two structural sources make this non-convexity *generic* rather than incidental. *Permutation symmetry*: relabeling the m hidden units of a layer (permuting rows of $\mathbf{W}_1$ and columns of $\mathbf{W}_2$ together) leaves the function unchanged, so every minimum comes in $m!$ copies; a function with $m!$ equivalent minima cannot be convex (a convex function’s minimizers form a single convex set, and the average of two permuted-copy minimizers is not generally a minimizer). *Scale symmetry*: with ReLU-family activations, $(c\mathbf{W}_1, \mathbf{W}_2/c)$ is function-preserving for $c > 0$, carving flat valleys through the landscape.

What rescues the enterprise is a redefinition of success. In the convex chapters, “solved” meant the global optimum. Here it means: *found parameters with low training loss that generalize* — and for that, the empirical situation (Goodfellow §8.2, refined by a decade of practice) is kinder than the worst case: in high dimensions, critical points with high loss are overwhelmingly *saddle points* rather than bad local minima (a local minimum requires *every* Hessian eigenvalue positive; at a random high-loss critical point, p independent-ish curvatures all positive is exponentially unlikely), and gradient methods with noise escape saddles. The practical enemy is not getting *trapped* — it is moving *slowly* across ill-conditioned plateaus and valley floors. That is why this chapter spends its effort on conditioning, not on global-optimality theory: the quadratic model problem of the next section is exactly the local picture of a valley floor, and everything it teaches transfers.

C.2 Gradient descent, analyzed exactly

C.2.1 The model problem and the eigenmode decomposition

Fix the strongly convex quadratic

$$f(\boldsymbol{\beta}) = \frac{1}{2}\boldsymbol{\beta}^\top \mathbf{A}\boldsymbol{\beta} - \mathbf{b}^\top \boldsymbol{\beta}, \quad \mathbf{A} \succ 0, \quad \nabla f(\boldsymbol{\beta}) = \mathbf{A}\boldsymbol{\beta} - \mathbf{b}, \quad \boldsymbol{\beta}^* = \mathbf{A}^{-1}\mathbf{b}. \quad (\text{C.2.1})$$

This is the right model problem three times over: it is the local Taylor picture of any smooth objective near a minimum (with $\mathbf{A}$ the Hessian); it is *exactly* least squares ($\mathbf{A} = \mathbf{X}^\top \mathbf{X}/N$); and it is

fully solvable, so every optimization phenomenon can be exhibited rather than asserted. Gradient descent iterates $\beta_{t+1} = \beta_t - \eta \nabla f(\beta_t)$, and the error $e_t = \beta_t - \beta^*$ obeys the exact linear recursion

$$e_{t+1} = (I - \eta A) e_t \quad \implies \quad e_t = (I - \eta A)^t e_0. \quad (\text{C.2.2})$$

Now diagonalize, $A = U\Lambda U^\top$ (Foundations A), and write the error in the eigenbasis, $\tilde{e}_{j,t} = u_j^\top e_t$:

$$\tilde{e}_{j,t} = (1 - \eta \lambda_j)^t \tilde{e}_{j,0}. \quad (\text{C.2.3})$$

Gradient descent decouples into p independent scalar geometric decays, one per curvature direction — precisely the normal-mode decomposition of coupled linear relaxation, with $1/\eta\lambda_j$ playing each mode's relaxation time. Everything in Sections C.2–C.5 is bookkeeping on (C.2.3):

- *Stability.* Every mode contracts iff $|1 - \eta\lambda_j| < 1$ for all j , i.e. $\eta < 2/\lambda_{\max}$: the stiffest direction sets the step-size ceiling. Above it, the stiff mode oscillates with growing amplitude — the divergence every practitioner has met as a loss curve exploding after a learning-rate increase.
- *Speed.* Within the stable range, the *slowest* mode dominates the asymptotics: the contraction factor is $\max_j |1 - \eta\lambda_j|$, governed by the soft direction $\lambda_{\min}$ once η is pinned by $\lambda_{\max}$. Exercise C.1 turns (C.2.3) into the syllabus's convergence statement, $f(\beta_t) - f^* \leq (1 - \eta\lambda_{\min})^t (f(\beta_0) - f^*)$ for $\eta \leq 1/\lambda_{\max}$ — *linear convergence* in the optimization sense: error decays geometrically, $\log(1/\varepsilon)$ iterations per decimal digit.
- *The general convex rate, for contrast.* Without strong convexity ($\lambda_{\min} = 0$ allowed: flat directions), the guarantee degrades from geometric to $f(\beta_t) - f^* = O(1/t)$ — stated here without proof (Boyd §9.3); the flat directions are never *wrong*, just never finished. Plateaus in neural training are this regime locally.

C.2.2 Conditioning: the number that runs the show

Optimize the tradeoff in (C.2.3): the best constant step balances the two extreme modes, $\eta^* = 2/(\lambda_{\max} + \lambda_{\min})$, giving per-step contraction

$$\rho^* = \frac{\kappa - 1}{\kappa + 1}, \quad \kappa = \frac{\lambda_{\max}}{\lambda_{\min}}, \quad \text{hence} \quad \text{\#iterations to } \varepsilon \sim O(\kappa \log(1/\varepsilon)) \quad (\text{C.2.4})$$

(Exercise C.1 again). The condition number is the entire story: $\kappa = 1$ (spherical bowl) converges in one step; $\kappa = 10^4$ needs ten thousand times the iterations, with the trajectory zigzagging across the narrow valley — each step overshoots the stiff direction while inching along the soft one. You have met κ twice before, and both encounters now pay off: Foundations A computed κ for ridge regression, $(\sigma_{\max}^2 + \lambda)/(\sigma_{\min}^2 + \lambda) \rightarrow 1$ as λ grows — *regularization is preconditioning*, one more job weight decay quietly performs; and Module 4 §4.1 showed correlated features make $X^\top X$ ill-conditioned — which is why standardized, decorrelated-ish features are not cosmetic but a direct multiplier on training speed.

For neural networks the Hessian is neither constant nor positive definite, but its empirical spectral signature is consistent (Goodfellow §8.2 and a large literature since): a bulk of near-zero eigenvalues — the flat, symmetry-induced directions of §C.1 — plus a few large outliers, i.e.

effective κ enormous by construction. Saddle points exist and are escapable; the chronic tax is this conditioning. The two families of repair are the subjects of §C.4 (reshape the *dynamics*: momentum) and §C.5 (reshape the *metric*: per-coordinate preconditioning), and the ideal both imitate is Newton’s method — multiply the gradient by A^{-1} and (C.2.2) converges in one step — which is unaffordable at scale and approximated from opposite ends by the two families.

Checkpoint exercise

Exercise C.1 [Syllabus: convergence and conditioning; merges syllabus items 1–2]. [★★★★ Core] Work on the model problem (C.2.1) with the eigenmode decomposition (C.2.3). (a) Show $f(\beta) - f^* = \frac{1}{2} e^\top A e = \frac{1}{2} \sum_j \lambda_j \tilde{e}_j^2$, so suboptimality is a λ -weighted sum of squared mode errors. (b) Prove the syllabus statement: for constant step $\eta \leq 1/\lambda_{\max}$ (so every factor $1 - \eta\lambda_j \in [0, 1)$), $f(\beta_t) - f^* \leq (1 - \eta\lambda_{\min})^{2t} (f(\beta_0) - f^*)$, and explain why the commonly quoted exponent t (rather than $2t$) is the same statement with a coarser bound. Identify where stability fails as $\eta \rightarrow 2/\lambda_{\max}$. (c) Optimize the worst-case contraction $\max_j |1 - \eta\lambda_j|$ over η : show the optimum equalizes the two extreme modes at $\eta^* = 2/(\lambda_{\max} + \lambda_{\min})$ with contraction $(\kappa - 1)/(\kappa + 1)$. (d) Conclude the iteration-complexity claim: reaching $f(\beta_t) - f^* \leq \varepsilon$ requires $t = O(\kappa \log(1/\varepsilon))$ iterations. (Hint: $\log \frac{\kappa-1}{\kappa+1} \approx -2/\kappa$ for large κ .)

C.3 Stochastic gradients: optimization under measurement noise

The full gradient of $f(\beta) = \frac{1}{N} \sum_i \mathcal{L}_i(\beta)$ costs a pass over all N observations; a *minibatch* of B indices gives the Monte Carlo estimate $\tilde{g} = \frac{1}{B} \sum_{i \in \text{batch}} \nabla \mathcal{L}_i(\beta)$, unbiased ($\mathbb{E}[\tilde{g}] = \nabla f$, the LLN logic of Foundations B §B.4.4) with covariance shrinking like $1/B$. SGD is gradient descent driven by this noisy measurement, and its behavior is the cleanest possible two-regime story, exhibited exactly on the model problem in Exercise C.2:

Transient regime. Far from the optimum the signal dominates the noise, and SGD contracts like its deterministic parent — same $(1 - \eta\lambda_j)$ mode-by-mode decay in expectation.

Noise-floor regime. Near the optimum the deterministic pull vanishes ($\nabla f \rightarrow 0$) but the noise does not, and the iterate settles into a stationary jitter around β^* — a *noise ball* whose expected squared radius scales as

$$\mathbb{E} \|\beta_\infty - \beta^*\|^2 \propto \frac{\eta \sigma_g^2}{\lambda_{\min}} \quad (\sigma_g^2 = \text{per-step gradient noise variance}), \quad (\text{C.3.1})$$

proportional to the step size: halve η and the asymptotic error halves, but the transient slows in exact compensation. This is a bias–variance tradeoff in η (Foundations B’s favorite decomposition, now in time rather than in model space), and it dictates every learning-rate schedule ever used: *large η early* (fast transient, who cares about the floor), *decay later* (lower the floor once you are near it). The classical conditions for exact convergence, $\sum_t \eta_t = \infty$, $\sum_t \eta_t^2 < \infty$ (Robbins–Monro), are the schedule version of the same arithmetic — enough total motion to get anywhere, decaying fast enough that the noise sums finitely. The language a physicist will prefer: (C.3.1) is a fluctuation–dissipation statement — η plays temperature, and annealing the learning rate is annealing, literally. The same noise has a celebrated upside: it is what kicks iterates off saddle

points and (more heuristically) biases trajectories toward flat minima; treat the second claim as folklore with supporting evidence, not theorem.

Finance reality check

Two financial corrections to deep-learning defaults. First, return panels are small by deep-learning standards (a few million rows, heavy noise), so the noise-ball term in (C.3.1) is genuinely binding — batch sizes and schedules deserve real tuning, and “train longer” saturates early because the floor, not the transient, is the constraint. Second, the i.i.d.-sampling assumption behind $\mathbb{E}[\tilde{g}] = \nabla f$ deserves a raised eyebrow: sampling minibatches uniformly across a decade of dates treats 2008 and 2017 as exchangeable — exactly the assumption the thesis disputes. Date-blocked batches, recency weighting, or regime-stratified sampling all change what “the” objective is; choose deliberately and document, because Module 5’s protocol will hold you to whatever you chose.

Checkpoint exercise

Exercise C.2 [Syllabus: the SGD noise ball]. [★★★★ Core] *Same model problem, but the update uses a noisy gradient: $\beta_{t+1} = \beta_t - \eta (\nabla f(\beta_t) + \xi_t)$ with $\mathbb{E}[\xi_t] = \mathbf{0}$, $\text{Cov}(\xi_t) = \sigma^2 \mathbf{I}$, independent across steps. (a) Show the error recursion is $e_{t+1} = (\mathbf{I} - \eta \mathbf{A})e_t - \eta \xi_t$, and take expectations: the mean error contracts exactly as in the noiseless case (C.2.3). (b) Derive the second-moment recursion per eigenmode: $\mathbb{E}[\tilde{e}_{j,t+1}^2] = (1 - \eta \lambda_j)^2 \mathbb{E}[\tilde{e}_{j,t}^2] + \eta^2 \sigma^2$, and solve for the stationary point $\mathbb{E}[\tilde{e}_{j,\infty}^2] = \eta^2 \sigma^2 / (1 - (1 - \eta \lambda_j)^2) \approx \eta \sigma^2 / (2 \lambda_j)$ for small η . (c) Sum over modes to obtain the noise-ball statement (C.3.1): $\mathbb{E} \|e_\infty\|^2 \approx \frac{\eta \sigma^2}{2} \sum_j \lambda_j^{-1}$, dominated by the softest mode $\lambda_{\min}$. State the bias–variance tradeoff in η precisely: which term in the full finite- t solution is “bias” (transient) and which “variance” (floor), and why decaying schedules get both. (d) One sentence each: what does (C.3.1) predict about doubling the batch size, and about why the loss curve of a real training run plateaus at a level that depends on the learning rate?*

C.4 Momentum: optimization as a damped oscillator

Heavy-ball momentum replaces the memoryless gradient step with an exponentially-weighted velocity:

$$v_{t+1} = \mu v_t - \eta \nabla f(\beta_t), \quad \beta_{t+1} = \beta_t + v_{t+1}, \quad (\text{C.4.1})$$

with momentum coefficient $\mu \in [0, 1)$ (the field’s notation is β ; we use μ to spare the regression coefficients). The right way to understand (C.4.1) is the continuous-time limit you can analyze in your sleep: it is the discretization of a *damped second-order system*,

$$\ddot{\beta} + \gamma \dot{\beta} + \nabla f(\beta) = 0 \quad \text{— a unit mass rolling on the loss surface with friction } \gamma. \quad (\text{C.4.2})$$

Exercise C.6 carries out the discretization both directions and identifies the dictionary: time step $h = \sqrt{\eta}$, friction $\gamma = (1 - \mu) / \sqrt{\eta}$ — so $\mu \rightarrow 1$ is the low-friction limit, and plain gradient descent ($\mu = 0$) is the *overdamped* limit in which inertia is negligible and velocity is always proportional to

force. Why does inertia help? Decompose (C.4.2) on the quadratic's eigenmodes: each mode is an independent damped harmonic oscillator with natural frequency $\sqrt{\lambda_j}$. The overdamped (GD) mode crawls at rate $\propto \lambda_j/\gamma$ — punishing for soft modes; an *underdamped* mode rings at frequency $\sqrt{\lambda_j}$ regardless of how small λ_j 's force constant is, reaching the bottom in time $\propto 1/\sqrt{\lambda_j}$ rather than $1/\lambda_j$. Tuning (μ, η) to put the slowest mode near *critical damping* while keeping the stiffest stable yields the classical result (Exercise C.6(c) verifies it on a single mode): optimal heavy-ball contraction is

$$\rho_{\text{HB}}^* = \frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \quad \text{— the condition number enters as } \sqrt{\kappa}, \quad (\text{C.4.3})$$

versus κ for plain GD (C.2.4): at $\kappa = 10^4$, a hundredfold reduction in iterations from one extra state vector. Nesterov's accelerated variant evaluates the gradient at the look-ahead point $\beta_t + \mu v_t$ and achieves the same $\sqrt{\kappa}$ dependence with cleaner worst-case guarantees; per the preface, we state and use, not prove. One honest caveat: the $\sqrt{\kappa}$ magic is a *deterministic* result; under heavy gradient noise momentum's averaging still helps (it is an EMA of gradients, variance-reducing), but the sharp acceleration story blurs — which is part of why the stochastic world drifted toward the methods of the next section.

C.5 Adaptive methods: AdaGrad, RMSProp, Adam

Momentum reshapes dynamics but still applies *one* learning rate to all coordinates. The adaptive family instead approximates Newton's preconditioner A^{-1} by something affordable: a *diagonal* matrix estimated from the gradients themselves. The lineage in three steps:

AdaGrad: accumulate $s_{j,t} = \sum_{\tau \leq t} g_{j,\tau}^2$ per coordinate and step $\Delta\beta_j = -\eta g_{j,t} / (\sqrt{s_{j,t}} + \epsilon)$. Coordinates with persistently large gradients get their rates cut; rarely-updated coordinates keep large rates. The defect for non-convex training: the accumulator only grows, so the effective rate decays to zero whether or not you have arrived.

RMSProp: replace the sum with an exponential moving average, $v_{j,t} = \beta_2 v_{j,t-1} + (1 - \beta_2) g_{j,t}^2$ — a forgetting window of effective length $1/(1 - \beta_2)$, so the preconditioner tracks the *current* local curvature scale rather than the whole history.

Adam: RMSProp's second-moment EMA plus a first-moment EMA (momentum), plus a correction:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^{\odot 2}, \quad \beta_{t+1} = \beta_t - \eta \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon}, \quad (\text{C.5.1})$$

with $\hat{\mathbf{m}}_t = \mathbf{m}_t / (1 - \beta_1^t)$ and $\hat{\mathbf{v}}_t = \mathbf{v}_t / (1 - \beta_2^t)$. The hats are the *bias correction*, and it is a Foundations B-style calculation, not a hack: an EMA initialized at zero is a biased estimator early on — unrolling gives $\mathbf{m}_t = (1 - \beta_1) \sum_{\tau=1}^t \beta_1^{t-\tau} \mathbf{g}_\tau$, whose weights sum to $1 - \beta_1^t < 1$, so for stationary gradients $\mathbb{E}[\mathbf{m}_t] = (1 - \beta_1^t) \mathbb{E}[\mathbf{g}]$; dividing by $(1 - \beta_1^t)$ restores unbiasedness (Exercise C.3 does both moments and quantifies when it matters: with the default $\beta_2 = 0.999$, $\mathbf{v}$ is badly undersized for hundreds of steps, and uncorrected Adam's effective early learning rate is silently enormous). Read the update as a whole: the step per coordinate is $\approx \eta \cdot \mathbb{E}[g_j] / \sqrt{\mathbb{E}[g_j^2]}$ — a *signal-to-noise-normalized* step, bounded near $\pm\eta$ regardless of gradient scale. That bounded-step property is

why Adam is forgiving of learning-rate choice and why it became the default for everything in this pipeline.

Two practicalities with theory inside. *AdamW*: adding an L_2 penalty to the loss and letting Adam adapt it is *not* weight decay — the penalty gradient $\lambda\beta$ gets divided by $\sqrt{\hat{v}}$ like everything else, so heavily-updated coordinates are barely regularized; AdamW instead applies decay directly to the parameters ($\beta \leftarrow (1 - \eta\lambda)\beta$, then the Adam step), restoring the clean MAP interpretation of Module 4 §4.5. Use AdamW when you mean weight decay. *Generalization debates*: a literature exists finding tuned SGD+momentum sometimes generalizes better than Adam; for the modest models of this pipeline the difference is second-order against data hygiene, and AdamW is the sensible default.

Checkpoint exercise

Exercise C.3 [Supplementary; discharges Module 6’s bias-correction exercise]. [★★★★ Core]
(a) Unroll the first-moment EMA $\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t$ from $\mathbf{m}_0 = \mathbf{0}$ to obtain $\mathbf{m}_t = (1 - \beta_1) \sum_{\tau=1}^t \beta_1^{t-\tau} \mathbf{g}_\tau$, and show the weights sum to $1 - \beta_1^t$ (geometric series). Conclude that for stationary gradients ($\mathbb{E}[\mathbf{g}_\tau] = \mathbf{g}$ for all τ), $\mathbb{E}[\mathbf{m}_t] = (1 - \beta_1^t) \mathbf{g}$, so $\hat{\mathbf{m}}_t = \mathbf{m}_t / (1 - \beta_1^t)$ is unbiased. Repeat for $\mathbf{v}_t$ with β_2 . (b) Quantify the early-iteration damage of skipping the correction: with the defaults $\beta_1 = 0.9$, $\beta_2 = 0.999$, evaluate the shrinkage factors $1 - \beta_1^t$ and $1 - \beta_2^t$ at $t = 10$ and $t = 100$. Derive the factor by which the uncorrected ratio $\mathbf{m}_t / \sqrt{\hat{v}_t}$ differs from the corrected one (express it in terms of the two shrinkage factors, then evaluate at $t = 10$), and state what an effective step of that size in the first dozen updates does to a freshly He-initialized network. (c) Show that bias correction becomes negligible for $t \gg 1/(1 - \beta_2)$, and give the numerical threshold for the defaults. (d) One sentence: using the chapter’s language, why is the corrected Adam step “SNR-normalized,” and what does that imply when gradient noise is heavy-tailed (Foundations B §B.7)?

C.6 Matrix calculus: the differential method

Backpropagation (Module 6) is the chain rule executed on matrix expressions, and the Matrix Cookbook is its lookup table. This section teaches the *one technique that generates every entry you will need*, so the Cookbook becomes a check rather than a crutch. The convention throughout: a gradient has the shape of its variable.

The method. For scalar $\mathcal{L}$, write the first-order perturbation (the *differential*) and read off the gradient from the canonical form:

$$d\mathcal{L} = \text{tr}(\mathbf{G}^\top d\mathbf{X}) \iff \nabla_{\mathbf{X}} \mathcal{L} = \mathbf{G}. \quad (\text{C.6.1})$$

Differentials obey the ordinary product and chain rules ($d(\mathbf{AB}) = (d\mathbf{A})\mathbf{B} + \mathbf{A}d\mathbf{B}$, $d\sigma(\mathbf{u}) = \sigma'(\mathbf{u}) \odot d\mathbf{u}$ elementwise), and the trace’s cyclic property (Foundations A) is the lever that isolates $d\mathbf{X}$. One worked specimen replaces a page of index gymnastics:

Example C.6.1 (The fundamental backprop identity). Let $\mathcal{L} = g(\mathbf{W}\mathbf{x})$ with $g : \mathbb{R}^m \rightarrow \mathbb{R}$, and write $\mathbf{y} = \mathbf{W}\mathbf{x}$, $\delta = \nabla_{\mathbf{y}} g$. Then

$$d\mathcal{L} = \delta^\top d\mathbf{y} = \delta^\top (d\mathbf{W}) \mathbf{x} = \text{tr}(\mathbf{x} \delta^\top d\mathbf{W}) = \text{tr}((\delta \mathbf{x}^\top)^\top d\mathbf{W}) \implies \nabla_{\mathbf{W}} \mathcal{L} = \delta \mathbf{x}^\top,$$

a rank-one outer product: $(\text{upstream sensitivity}) \times (\text{local input})^\top$. Every weight gradient in every layer of every network in this syllabus has this shape; Exercise C.4 (the syllabus’s matrix-chain-rule exercise) re-derives it from first principles and extends it through deeper compositions, where the same move yields $\nabla_x \mathcal{L} = \mathbf{W}^\top \delta$ — the signal that propagates backward.

Jacobians, JVPs, VJP. For vector-valued $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$, the Jacobian J is $m \times n$; the chain rule composes Jacobians, but *materializing* them is almost never affordable or necessary. Automatic differentiation computes one of two matrix-free products: forward mode pushes a tangent through (Jv , a JVP, one pass per *input* direction), reverse mode pulls a sensitivity back ($J^\top u$, a VJP, one pass per *output*). A training loss is a scalar — one output, millions of inputs — so reverse mode computes the entire gradient in a single backward pass, at the price of storing forward activations. That asymmetry is backpropagation, and Exercise C.7 makes the cost accounting precise. (You met the pattern before the name: the softmax–cross-entropy cancellation of Module 4 §4.10 is a VJP through the softmax Jacobian, simplified analytically.)

Identities for the road (derivable by (C.6.1) in a line or two; Cookbook §2.4–2.7): $\nabla_x (a^\top x) = a$ and $\nabla_x (x^\top A x) = (A + A^\top)x$ — both already proven in Foundations A, now re-derivable in one breath; $\nabla_X \text{tr}(A^\top X) = A$; and, stated for Module 8 where it powers the Gaussian-mixture M-step: $\nabla_A \log |A| = A^{-\top}$.

Checkpoint exercise

Exercise C.4 [Syllabus: the matrix chain rule for backprop]. [★★★★ Core] (a) From first principles (no differentials yet): for $\mathcal{L}(\mathbf{W}) = g(\mathbf{W}\mathbf{x})$ with $g : \mathbb{R}^m \rightarrow \mathbb{R}$, compute $\partial \mathcal{L} / \partial W_{ij}$ by the scalar chain rule and assemble the result into $\nabla_{\mathbf{W}} \mathcal{L} = (\nabla_y g) \mathbf{x}^\top$ where $\mathbf{y} = \mathbf{W}\mathbf{x}$. (b) Re-derive the same identity in two lines with the differential method (C.6.1), and additionally extract $\nabla_x \mathcal{L} = \mathbf{W}^\top (\nabla_y g)$ from the same differential. (c) Generalize to a deeper composition $\mathcal{L} = g(\mathbf{W}_2 \sigma(\mathbf{W}_1 \mathbf{x}))$ with elementwise σ : derive $\nabla_{\mathbf{W}_2} \mathcal{L}$, $\nabla_{\mathbf{W}_1} \mathcal{L}$, and observe the recursive structure — each weight gradient is (backpropagated sensitivity at its output) $\times$ (its input) $^\top$, with sensitivities updated by $\delta \leftarrow \mathbf{W}^\top \delta$ and $\delta \leftarrow \sigma'(\mathbf{u}) \odot \delta$ alternately. (You have just derived backpropagation; Module 6 will only add bookkeeping and biases.) (d) Use the differential method to verify two Foundations A identities in one line each: $\nabla_x (x^\top A x) = (A + A^\top)x$ and $\nabla_X \text{tr}(A^\top X) = A$.

C.7 Initialization: variance propagation through depth

Why does anyone care how random initial weights are scaled? Because depth turns small scale errors into exponential catastrophes, by an argument that is pure Foundations B variance algebra. Consider a linear layer $\mathbf{h} = \mathbf{W}\mathbf{x}$ with n_{in} inputs, weights i.i.d., zero-mean, variance σ_W^2 , independent of the zero-mean inputs. Then each output coordinate is a sum of n_{in} independent products, and (Foundations B, eq. B.1.4-style algebra)

$$\text{Var}(h_i) = n_{\text{in}} \sigma_W^2 \text{Var}(x_j) \implies \text{variance gain per layer} = n_{\text{in}} \sigma_W^2. \quad (\text{C.7.1})$$

Stack L layers and the signal variance scales as $(n \sigma_W^2)^L$: the same geometric amplification logic as the GD modes (C.2.3), and the same one that will reappear as the RNN vanishing/exploding-gradient theorem in Module 7 (ρ^T on the recurrent Jacobian — one mechanism, three costumes).

A gain of 1.1 per layer is a factor 117 at depth 50; a gain of 0.9 is 0.005. The *only* survivable choice at depth is gain = 1, and the same argument run on the *backward* pass (gradients flow through $W^\top$, so n_{out} replaces n_{in}) gives a second, generally incompatible condition. The classical resolutions, whose full derivations are Module 6 exercises by syllabus design (this section supplies the framework; Module 6 supplies the activations' bookkeeping): *Xavier/Glorot*, the compromise $\sigma_W^2 = 2/(n_{\text{in}} + n_{\text{out}})$ for symmetric saturating activations; *He*, $\sigma_W^2 = 2/n_{\text{in}}$ for ReLU — the factor 2 because ReLU zeroes half its inputs, halving the variance that (C.7.1) must replenish. Exercise C.5 runs the recursion numerically so the exponential is felt, not believed. Batch normalization (Module 6) attacks the same problem at runtime — renormalize per layer and let the network learn the scales — which is why init discipline and normalization layers are partial substitutes.

Checkpoint exercise

Exercise C.5 [Supplementary: feeling the exponential]. [★★★★ Core] *Take the variance recursion (C.7.1) for a stack of L linear layers of equal width $n = 512$, inputs standardized to unit variance. (a) Compute the output standard deviation at $L = 50$ for three choices of σ_W^2 : the gain-one choice $1/n$, and the mis-scaled choices $1.1/n$ and $0.9/n$. (b) Run the same recursion on the backward pass and state the condition under which forward and backward gain-one conditions conflict; verify the Xavier compromise $\sigma_W^2 = 2/(n_{\text{in}} + n_{\text{out}})$ is the harmonic-mean resolution. (c) For ReLU, accept the stated fact that the activation halves the variance and show the He choice $\sigma_W^2 = 2/n_{\text{in}}$ restores gain one. (The careful derivation of the factor $\frac{1}{2}$ — the half-Gaussian second moment — is a Module 6 exercise by syllabus design; here you only use it.) (d) Connect in two sentences to the two other geometric amplifications met in this book: the GD mode decay (C.2.3) and the $\rho^\top$ recurrent-Jacobian product previewed for Module 7. What single mathematical fact underlies all three?*

C.8 Capstone: reading a training configuration like a physicist

The corrected NEC will be trained by something like: *AdamW, lr 10^{-3} with cosine decay, batch 256 (date-blocked), gradient-norm clip 1.0, weight decay 10^{-2} on everything except the gate (which gets more), He init, 50–200 epochs with early stopping on purged-validation rank-IC*. Every clause is now a theorem citation:

- *AdamW, lr 10^{-3}* : bounded SNR-normalized steps (C.5.1) against an ill-conditioned spectrum (§C.2.2); decay applied as true weight decay so Module 4's MAP reading survives the preconditioner (§C.5).
- *Cosine decay*: the two-regime story of §C.3 — big steps for the transient, small for the noise floor (C.3.1); any decreasing schedule respecting Robbins–Monro spirit does the job, cosine is a smooth convention.
- *Batch 256, date-blocked*: $1/B$ gradient-noise scaling, with blocks because uniform date sampling asserts the stationarity the thesis denies (§C.3 box).
- *Clip 1.0*: insurance against the heavy-tailed gradient spikes that heavy-tailed *data* (Foundations B §B.7) inject through the loss, and against the recurrent amplification (C.7.1)-style that

Module 7 formalizes for the sequence encoder.

- *Weight decay, extra on the gate*: conditioning repair (§C.2.2) and the separation cure of Module 4 §4.8.3 — the gate is the component that can run away.
- *He init*: gain-one variance propagation (C.7.1) for ReLU experts; the GRU/LSTM encoder uses its own gain conventions for the same reason.
- *Early stopping on purged-validation rank-IC*: optimization halted by Module 5’s judge, not by training loss — the one knob that optimizes the thing actually being graded.

Checkpoint (from the syllabus). Closed book: analyze gradient descent’s convergence on a strongly convex quadratic (eigenmodes, stability ceiling $2/\lambda_{\max}$, rate (C.2.4)); derive Adam from scratch — both EMAs, both bias corrections, and why the corrected step is SNR-normalized; and explain why initialization matters in the language of variance propagation (C.7.1), including what goes wrong geometrically at depth. If those three are comfortable, you are equipped to derive backpropagation and analyze its failure modes in Module 6.

C.9 Exercises

Importance labels. Each exercise carries a priority badge for the NEC/MoE thesis track, reflecting how load-bearing it is for training and debugging the network — *not* how hard it is. [★★★ Core] are the toolkit you cannot train the NEC without: gradient-descent dynamics (C.1), the stochastic noise floor that sets batch size and schedule (C.2), the Adam optimizer (C.3), the backpropagation chain rule (C.4), and variance-preserving initialization (C.5). [★★ Worth it] are high-value enrichment a notch below: momentum’s damped-oscillator analysis (C.6), which Adam subsumes operationally but which a physicist will find the most natural exercise in the chapter, and the reverse-mode cost accounting (C.7). The one piece of background assumed throughout is the eigendecomposition of a symmetric matrix ($A = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^\top$); the chapter bridges to it through normal modes and damped oscillators rather than presuming fluency.

Questions only, as established. The syllabus’s five analytical exercises all appear (the first two merged into one multi-part problem, since the second is the natural continuation of the first); three supplementary exercises (marked) complete loops the chapter opened. Note on de-duplication: the momentum-ODE and SGD-quadratic derivations are listed by the syllabus under both this module and Module 6, and Adam’s bias correction under Module 6 — all three live *here*, once; the Module 6 chapter will cross-reference. Solvability audit: each exercise names its in-text tools. **Core exercises ([★★★ Core]) sit inline in the chapter body as checkpoint exercises, at the end of the section they test;** this section collects the supplementary exercises only.

On momentum (Section C.4)

Exercise C.6 [Syllabus: momentum as a damped ODE]. [★★ Worth it] (a) Discretize (C.4.2) with time step h : central second difference $\ddot{\beta} \rightarrow (\beta_{t+1} - 2\beta_t + \beta_{t-1})/h^2$ and a backward difference $\dot{\beta} \rightarrow (\beta_t - \beta_{t-1})/h$ for the friction term (the choice that makes the dictionary exact — check that a forward

difference gives the same dictionary only to first order in h , and rearrange into the heavy-ball form (C.4.1). Derive the dictionary: $\mu = 1 - \gamma h$ and $\eta = h^2$, i.e. $h = \sqrt{\eta}$, $\gamma = (1 - \mu) / \sqrt{\eta}$. (b) Interpret the limits through the dictionary: what friction regime is $\mu = 0$ (plain GD)? What happens physically as $\mu \rightarrow 1$ at fixed η , and why does practice cap μ near 0.9–0.99? (c) ★ Single-mode analysis: on one eigenmode with curvature λ , heavy-ball is the linear two-step recursion $\tilde{e}_{t+1} = (1 + \mu - \eta\lambda)\tilde{e}_t - \mu\tilde{e}_{t-1}$. Write it as a 2×2 companion matrix, compute the eigenvalues, and show that choosing $\mu = (1 - \sqrt{\eta\lambda})^2$ places the mode at critical damping (a double root) with contraction $1 - \sqrt{\eta\lambda}$. Conclude that tuning (η, μ) for the extreme modes simultaneously yields the $\sqrt{\kappa}$ rate (C.4.3). (This is the underdamped-oscillator analysis of your mechanics courses wearing optimization clothes; the ★ marks bookkeeping, not depth.)

On matrix calculus (Section C.6)

Exercise C.7 [Supplementary: why reverse mode]. [★★ **Worth it**] Consider a chain $\mathbf{x} \in \mathbb{R}^n \mapsto \mathbf{h}_1 \in \mathbb{R}^{m_1} \mapsto \dots \mapsto \mathbf{h}_L \in \mathbb{R}^{m_L} \mapsto \mathcal{L} \in \mathbb{R}$, with Jacobians $\mathbf{J}_\ell$ of the ℓ -th map. (a) The gradient is $\nabla_{\mathbf{x}} \mathcal{L} = \mathbf{J}_1^\top \mathbf{J}_2^\top \dots \mathbf{J}_L^\top \nabla_{\mathbf{h}_L} \mathcal{L}$. Count the cost of evaluating this product right-to-left (reverse mode: a sequence of VJPs) versus left-to-right (forward mode: JVPs seeded with each of the n basis vectors), in terms of n , L , and the layer widths. Conclude that reverse mode computes the full gradient at $O(1)$ times the cost of one forward pass, while forward mode pays a factor n . (b) Identify reverse mode’s price: which intermediate quantities must be stored between the forward and backward passes, and how does that memory scale with depth and width? (This is why sequence models truncate backpropagation through time — Module 7’s opening problem.) (c) When would forward mode win? State the shape condition, and give one example from this pipeline’s neighborhood (e.g. sensitivity of a scalar-input simulation).

Source map and what comes next

Section	Primary source(s)	Feeds directly into
C.1 Convexity, non-convexity	Boyd Ch. 1, §3.1–3.2; GF §8.2	expectations for NN training
C.2 GD on quadratics, rates	Boyd §9.1–9.3	every training run; lr selection
C.2.2 Conditioning	GF §4.3, §8.2	feature standardization; why precondition
C.3 SGD, noise ball, schedules	GF §8.1, §8.3	batch/schedule choices (Module 6)
C.4 Momentum as damped ODE	GF §8.3 + classical	optimizer intuition; Nesterov
C.5 AdaGrad/RMSProp/Adam(W)	GF §8.5 context	the pipeline’s default optimizer
C.6 Differential method, VJPs	Matrix Cookbook §2.4–2.7	backprop (Module 6); GMM M-step (Module 8)
C.7 Variance propagation	GF §8.4 context	Xavier/He derivations (Module 6); BPTT (Module 7)
C.8 Config capstone	synthesis	the NEC training loop

Module 6 is next and inherits a running start: backpropagation is already nine-tenths derived in Exercise C.4(c), the SGD/momentum/Adam analyses live here and will be cited rather than repeated, and the initialization framework of §C.7 needs only the activation-specific bookkeeping (the ReLU half-variance, the batch-norm gradient) that the syllabus assigns there. What Module 6 adds that is genuinely new: loss-function choices for regression on heavy-tailed targets, the regularization toolkit (dropout, early stopping as implicit shrinkage), batch normalization’s

mechanics and its gradient, and the practical craft of training and debugging the MLP experts that the corrected NEC will route between.